Deep Learning: Deep Neural Network

Part – 1

Dr. Oybek Eraliev,

Department of Computer Engineering

Inha University In Tashkent.

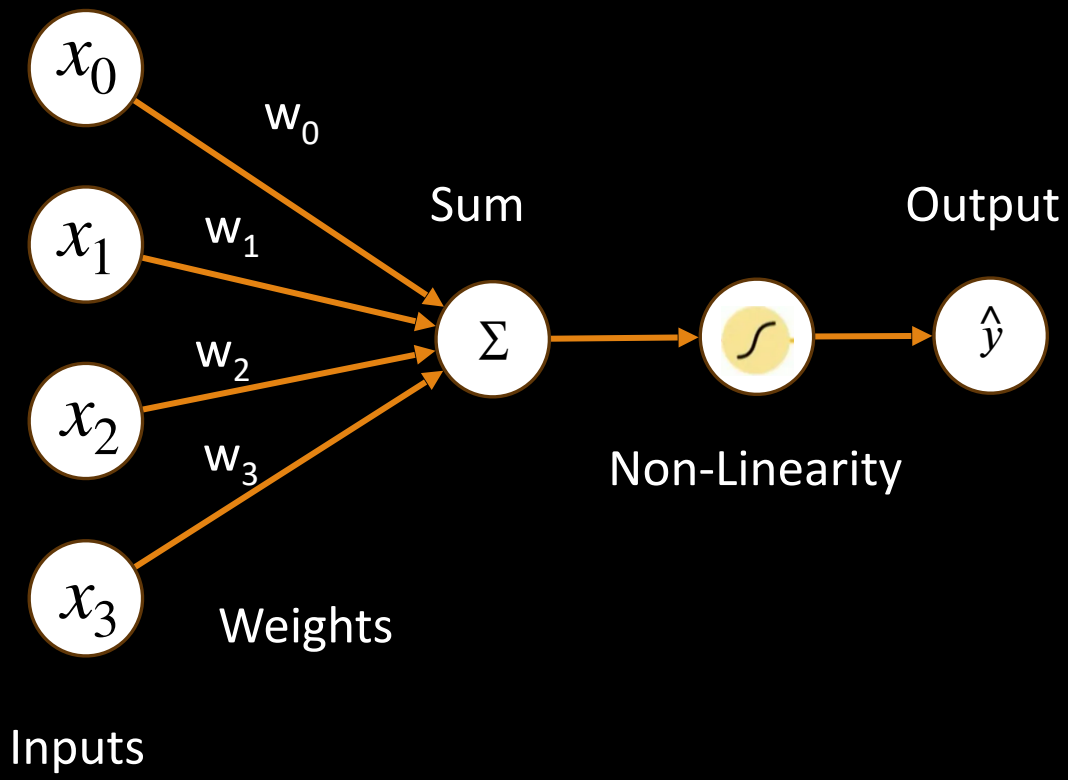
Email: oybekeraliev7@gmail.com

Content

- Model representation of Artificial Neural Network
- Loss Functions of Neural Network
- Backpropagation
- Activation Functions
- Implementation of DNN

Model representation of Artificial Neural Network

The Perceptron: Forward Propagation

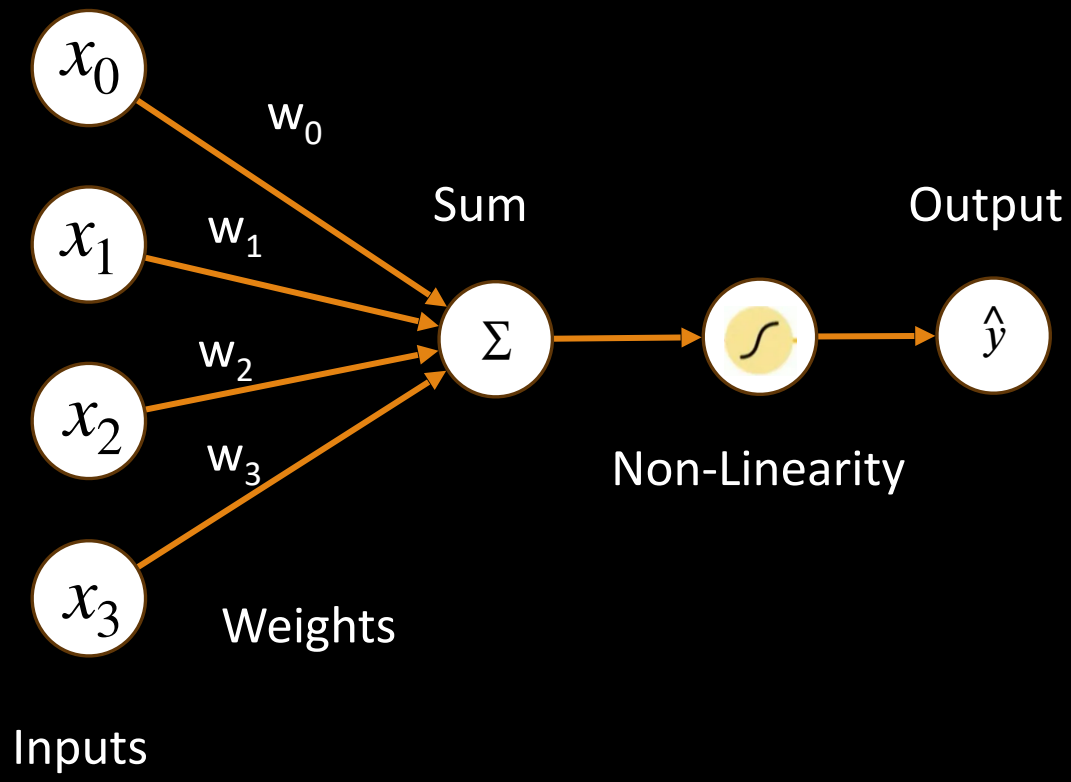


$$\hat{y} = g(w_0 + \sum_{i=1}^{m} x_i w_i)$$

g is non-linear activation function

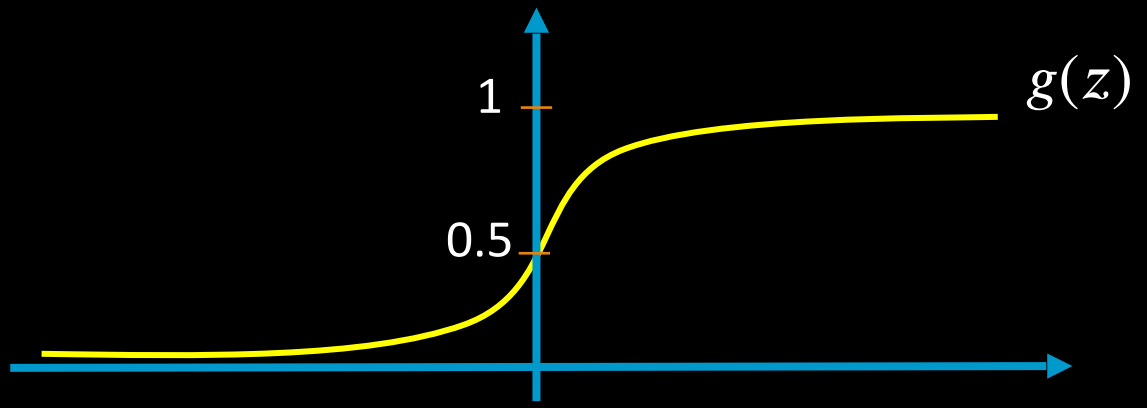
Model representation of Artificial Neural Network

The Perceptron: Forward Propagation



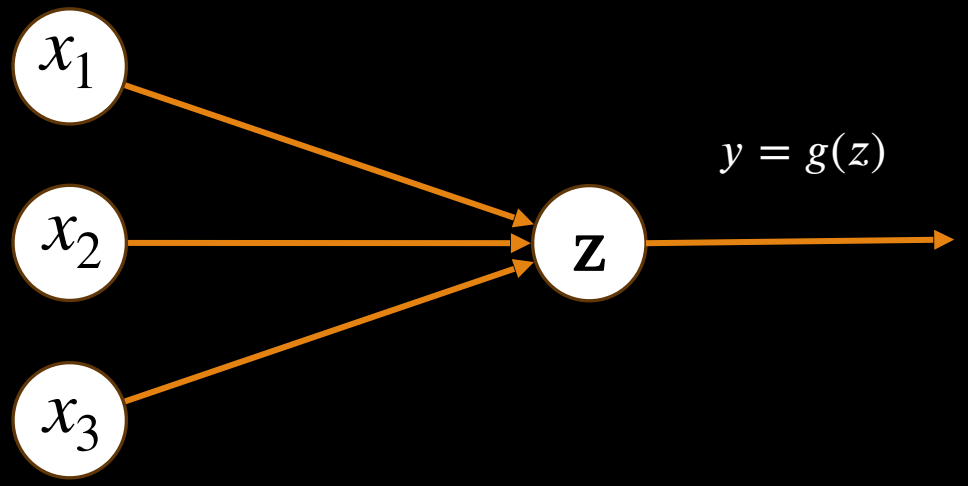
Activation function

$$g(z) = \frac{1}{1 + e^{-z}}$$



Model representation of Artificial Neural Network

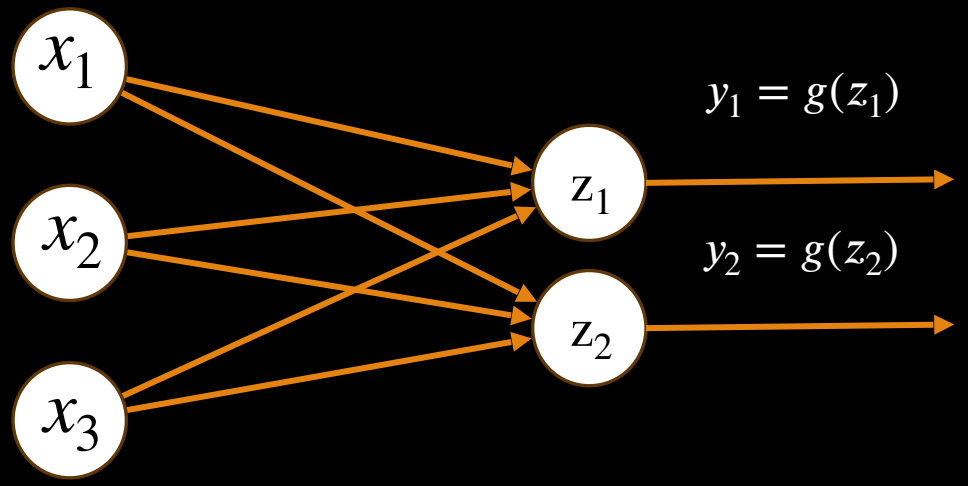
The Perceptron: Simplified



$$y = g(w_0 + \sum_{j=1}^{m} x_j w_j)$$

Model representation of Artificial Neural Network

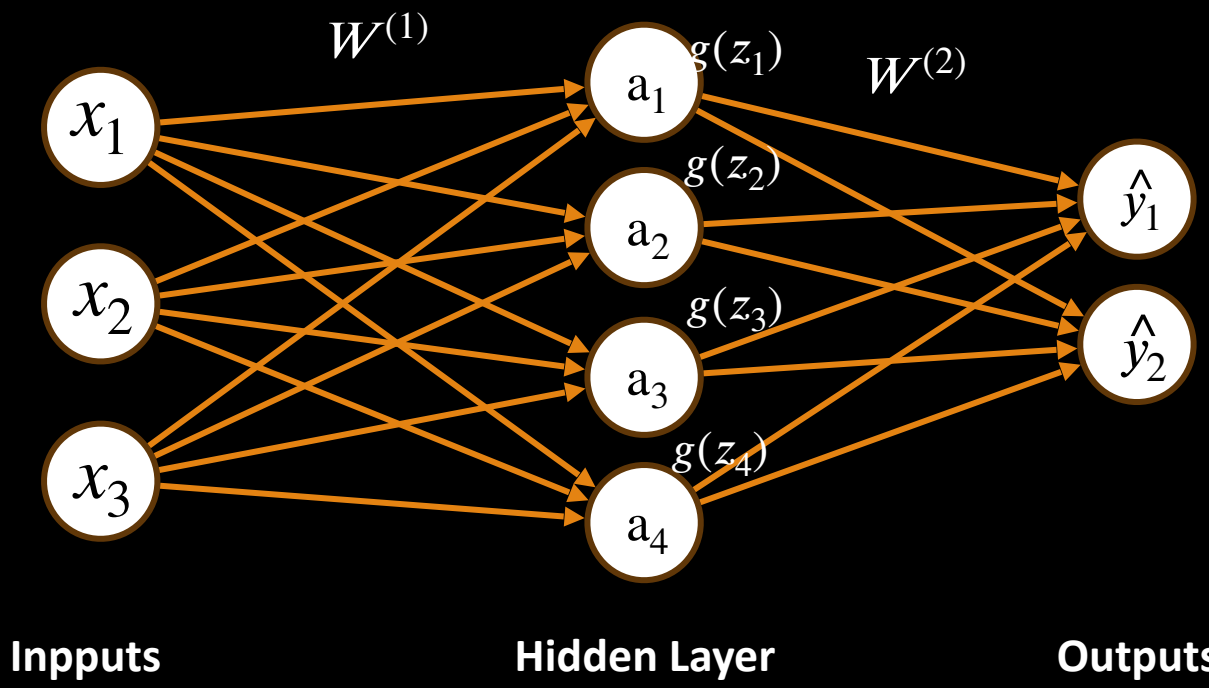
Multi Output Perceptron



$$z_i = g(w_{0,i} + \sum_{j=1}^{m} x_j w_{j,i})$$

Model representation of Artificial Neural Network

Single Layer Neural Network



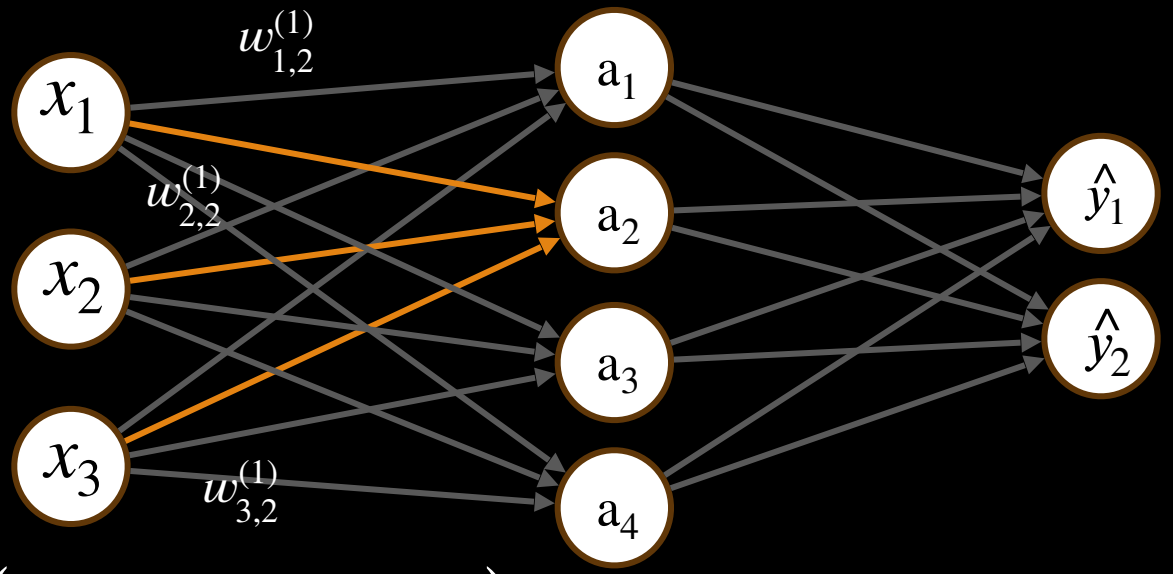
$$z_i = w_{0,i}^{(1)} + \sum_{j=1}^{m} x_j w_{j,i}^{(1)}$$

$$a_i = g(z_i)$$

$$\hat{y}_i = g(w_{0,i}^{(2)} + \sum_{i=1}^{m} g(z_j) w_{j,i}^{(2)})$$

Model representation of Artificial Neural Network

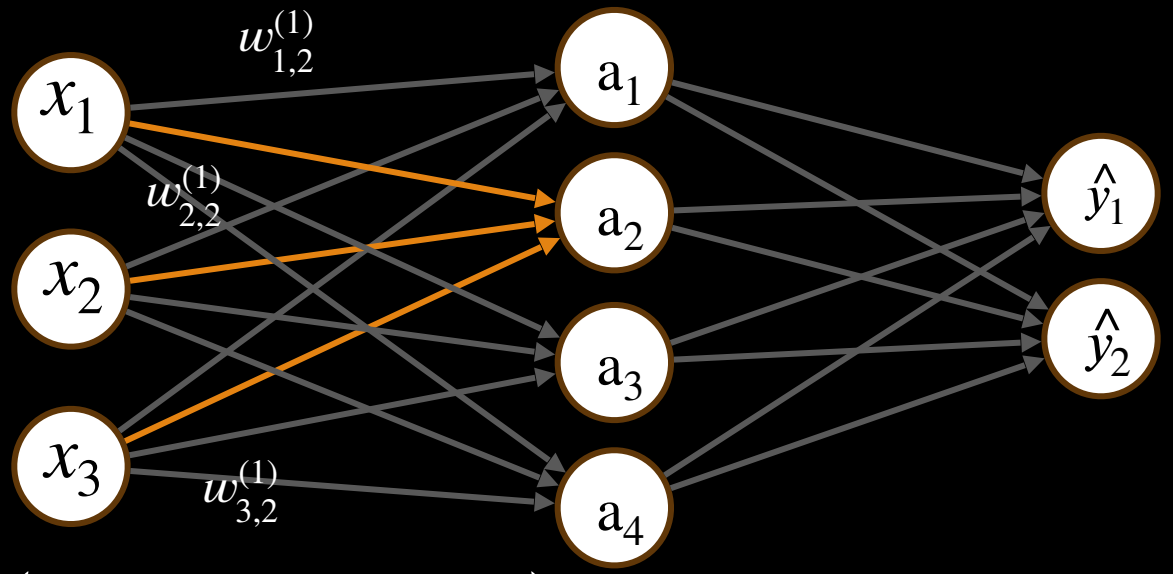
Single Layer Neural Network



$$a_2 = g\left(b^{(1)} + \sum_{j=1}^{m} x_j w_{j,2}^{(1)}\right) = g(b^{(1)} + x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + x_3 w_{3,2}^{(1)})$$

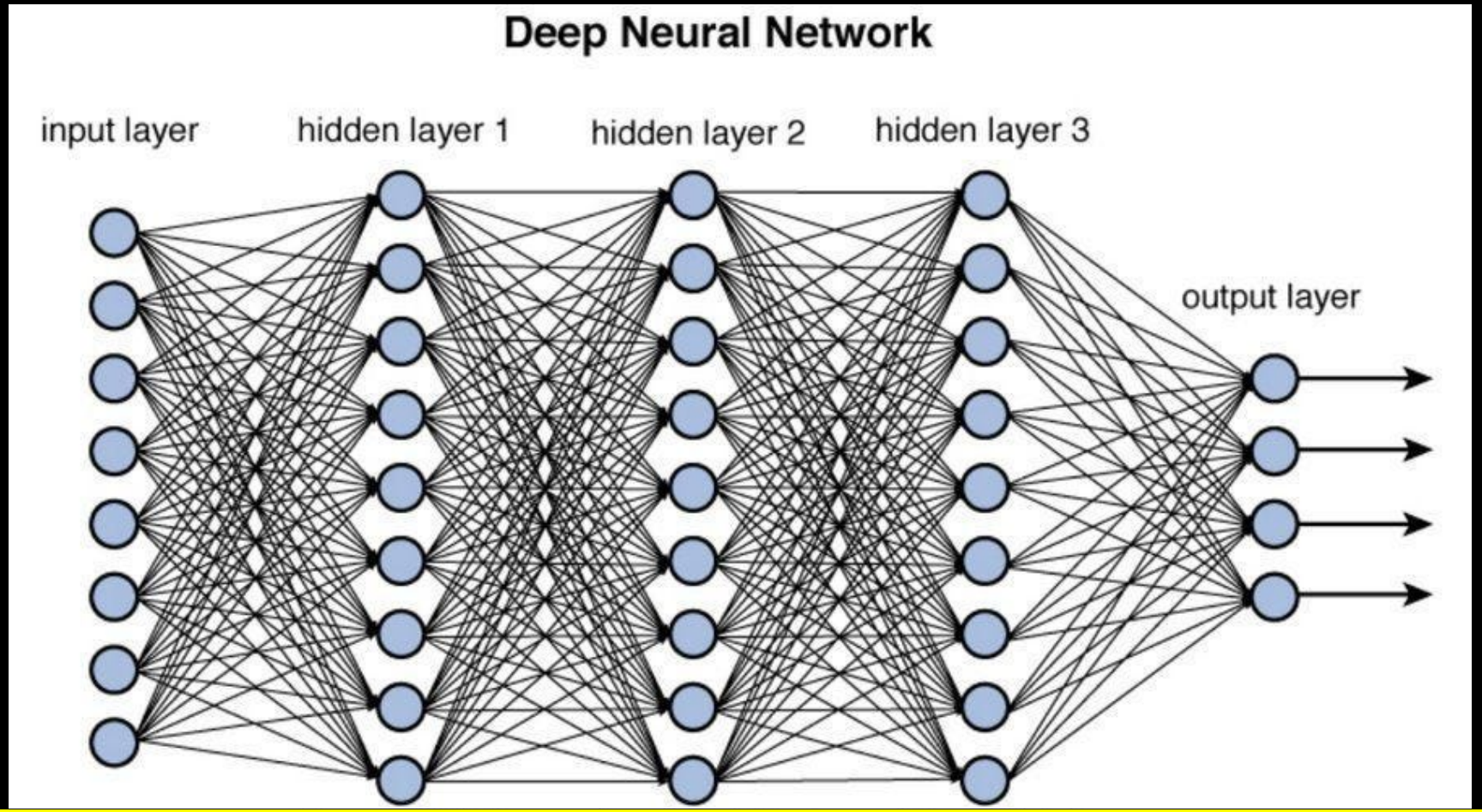
Model representation of Artificial Neural Network

Single Layer Neural Network



$$a_2 = g\left(w_{0,2}^{(1)} + \sum_{j=1}^{m} x_j w_{j,2}^{(1)}\right) = w_{0,2}^{(1)} + x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + x_3 w_{3,2}^{(1)}$$

Model representation of Artificial Neural Network

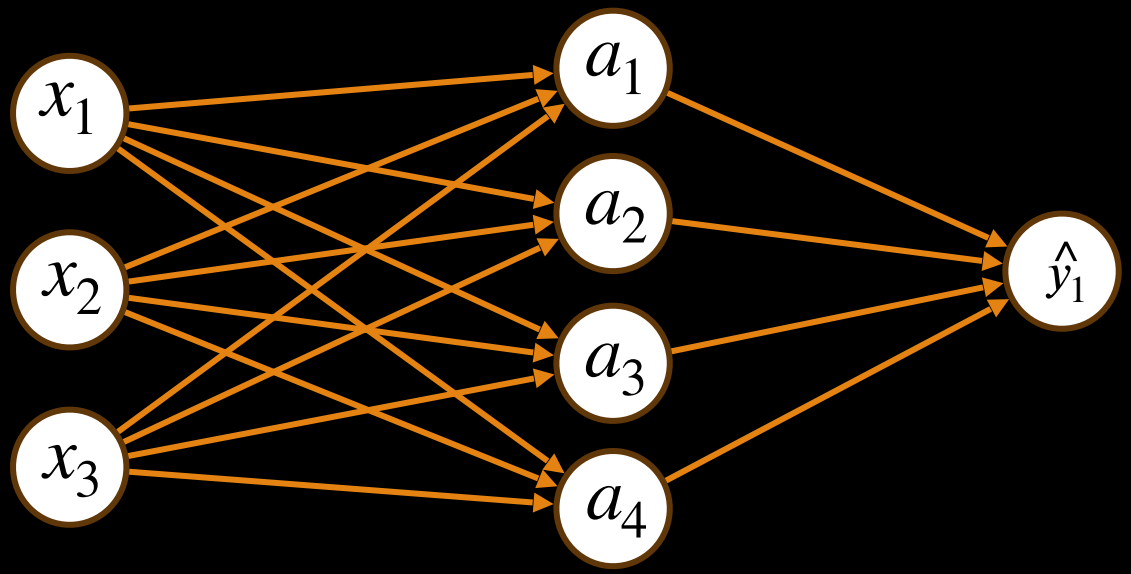


Content

- ~~Model representation of Artificial Neural Network~~
- Loss Functions of Neural Network
- Backpropagation
- Activation Functions
- Implementation of DNN

Loss Functions of Neural Network

Binary Cross Entropy Loss

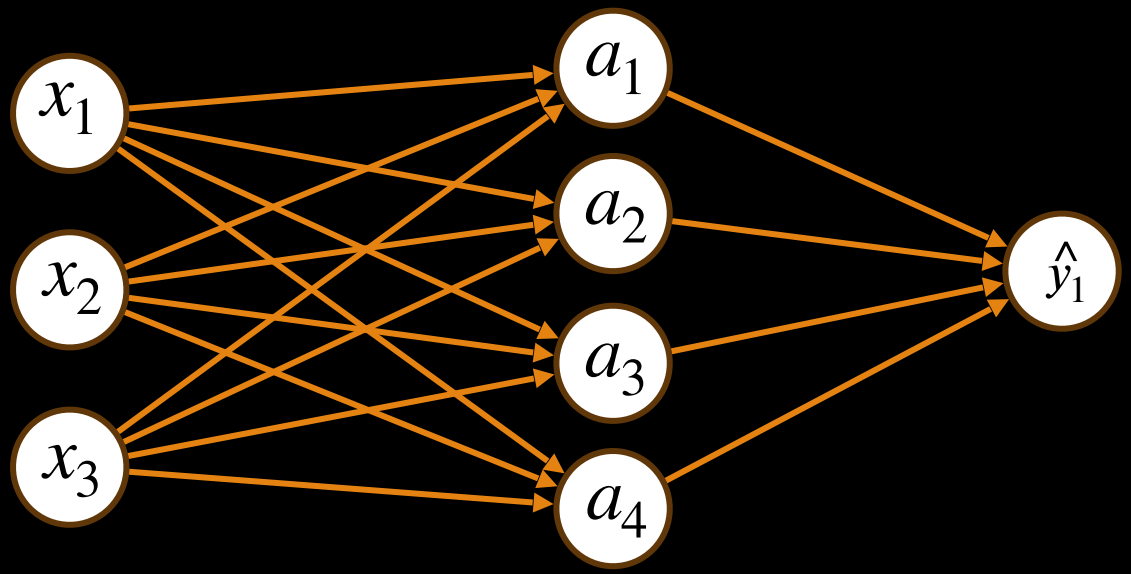


Cross entropy loss can bu used with the models that output a probability between 0 and 1

$$J(W) = -\frac{1}{m}\sum_{i=1}^{m} y^{(i)}\log\left(\hat{y}^{(i)}\right) + (1 - y^{(i)})\log\left(1 - \hat{y}^{(i)}\right)$$

Loss Functions of Neural Network

Mean Squared Error Loss



Mean squared error loss can bu used with regression models that output continuous real numbers

$$J(W) = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)})^2$$

Content

- ~~History and Intro to Deep Learning~~
- ~~Model representation of Artificial Neural Network~~
- ~~Loss Functions of Neural Network~~
- Backpropagation
- Activation Functions
- Implementation of DNN

Backpropagation

Optimization (Gradient Descent)

Algorithm:

1. Initialize wights randomly W
2. Loop until convergence:
    1. Compute Gradient: $\frac{\partial J(W)}{\partial W}$
    2. Update weights: $W := W - \alpha \frac{\partial J(W)}{\partial W}$
3. Return weights

Backpropagation

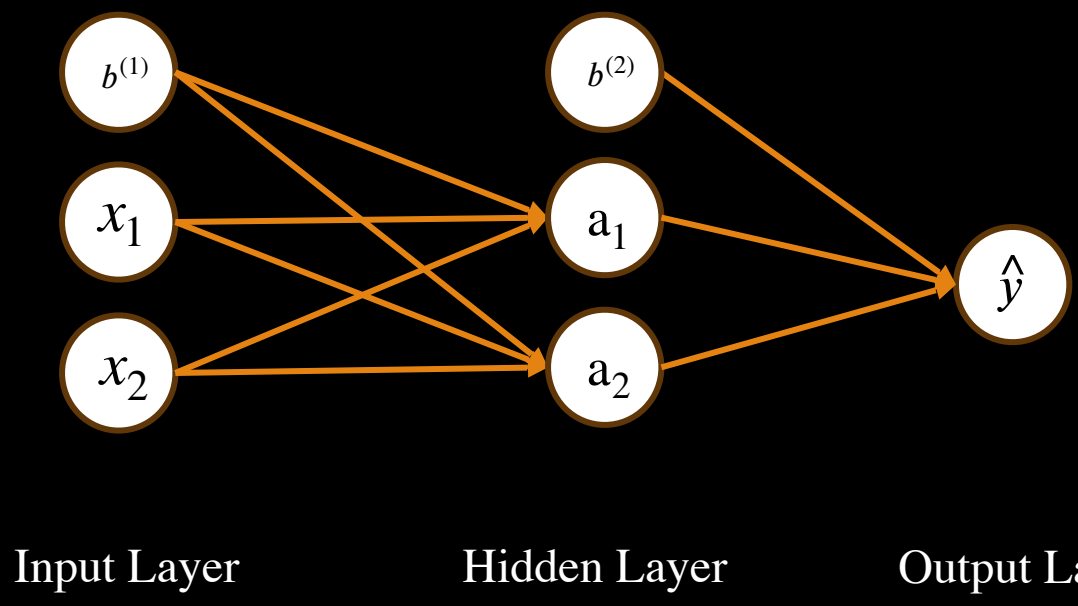
Forward Propagation of DNN

Network Architecture:

1.Input Layer: $x1$, $x2$

2.Hidden Layer 1: a1, a2 (two neurons)

3.Output Layer: $\hat{y}$ (one neuron)



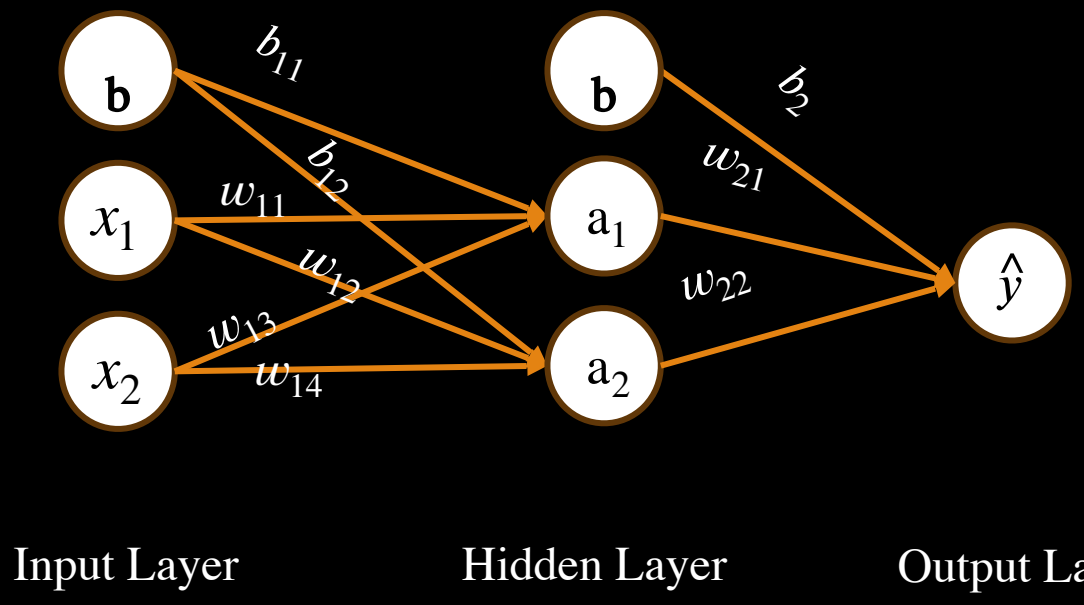
Backpropagation

Forward Propagation of DNN

Weights and Biases:

•Input to Hidden Layer:
- Weights:, $w_{11} w_{13}$ for a_1; w_{12}, w_{14} for a_2
- Biases: b_{11}, b_{12}

•Hidden Layer to Output Layer:
- Weights: w_{21}, w_{22} for $\hat{y}$
- Bias: b_2

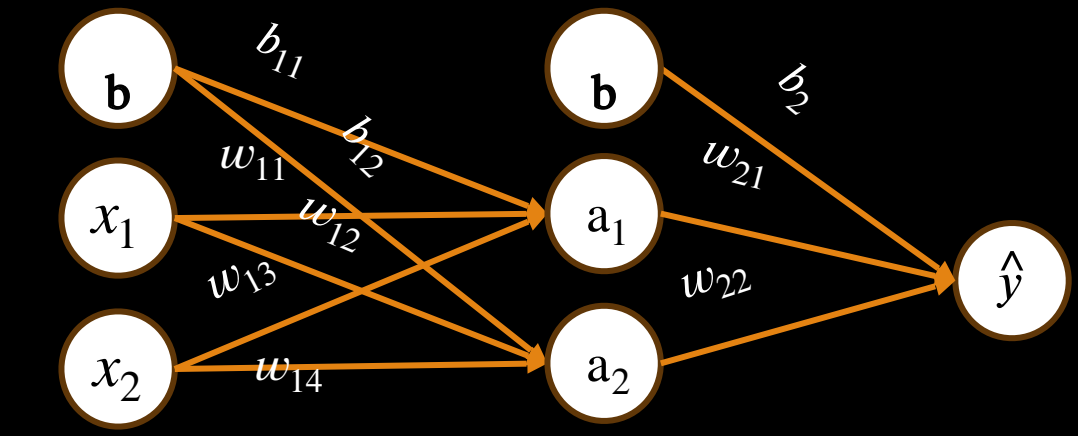


Backpropagation

Forward Propagation of DNN

Step 1: Input to Hidden Layer
The activations h_{11} and h_{12} are calculated as:
$a_1 = g(x_1 w_{11} + x_2 w_{13} + b_{11})$
$a_2 = g(x_1 w_{12} + x_2 w_{12} + b_{12})$

Step 2: Hidden Layer to Output Layer
The output $\hat{y}$ is calculated as:
$\hat{y} = g(a_1 w_{21} + a_2 w_{22} + b_2)$



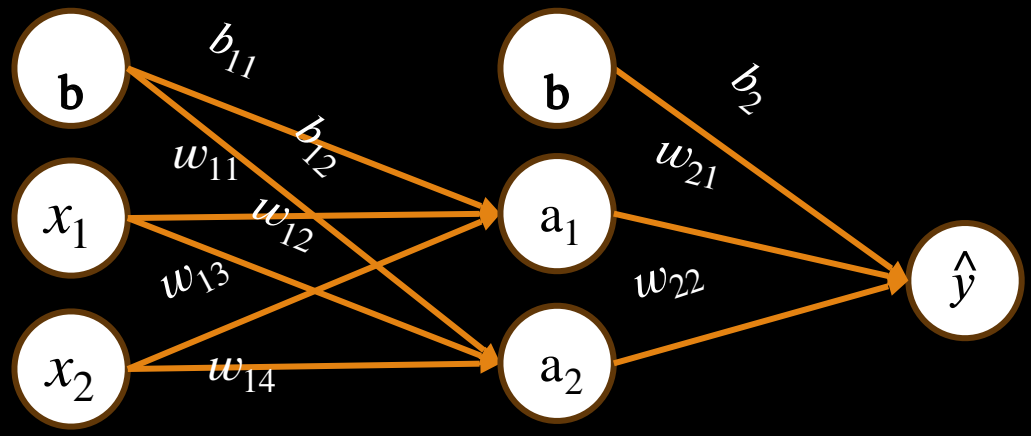
Backpropagation

Backpropagation of DNN

Neural network for Regression problem

Step 1: Compute the Loss
Assume the loss function is Mean Squared Error (MSE):

$$Loss = (\hat{y} - y)^2$$

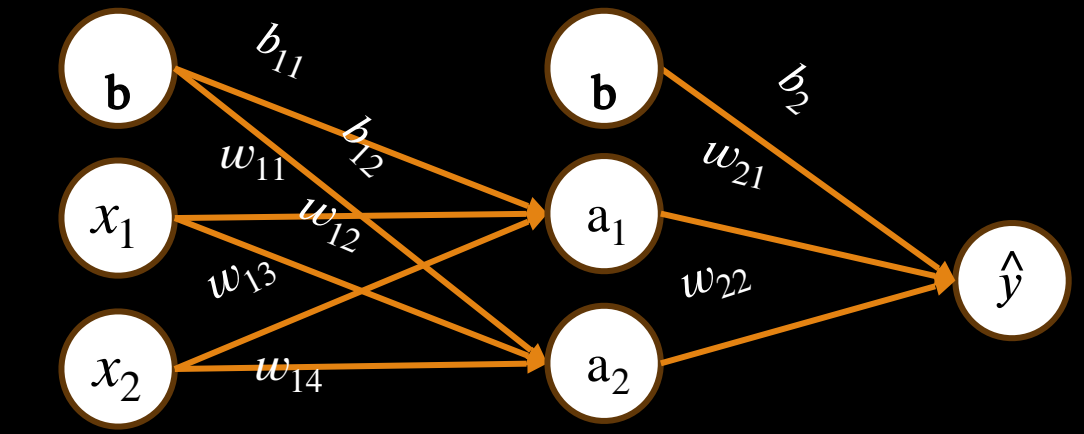


Backpropagation

Backpropagation of DNN

Step 2: Output Layer Gradients
The derivative of the loss with respect to the output is:

$$\frac{\partial Loss}{\partial \hat{y}} = 2(\hat{y} - y)$$



Backpropagation

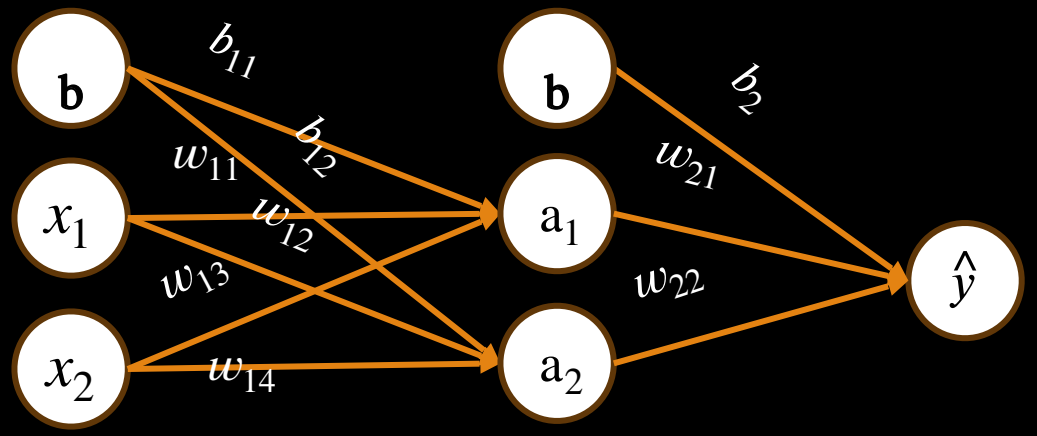
Backpropagation of DNN

Step 3: Hidden Layer weights Gradients

$$\frac{\partial Loss}{\partial w_{21}} = \frac{\partial Loss}{\partial \hat{y}} \bullet \frac{\partial \hat{y}}{\partial w_{21}}$$

$$\frac{\partial Loss}{\partial w_{22}} = \frac{\partial Loss}{\partial \hat{y}} \bullet \frac{\partial \hat{y}}{\partial w_{22}}$$

$$\frac{\partial Loss}{\partial b_2} = \frac{\partial Loss}{\partial \hat{y}} \bullet \frac{\partial \hat{y}}{\partial b_2}$$



Backpropagation

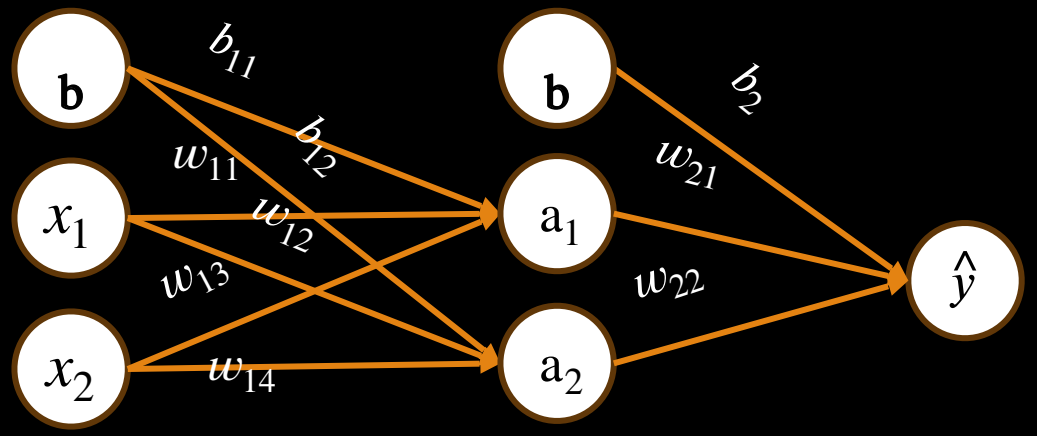
Backpropagation of DNN

Step 4: Input Layer weights Gradients

$$\frac{\partial Loss}{\partial w_{11}} = \frac{\partial Loss}{\partial \hat{y}} \bullet \frac{\partial \hat{y}}{\partial a_1} \bullet \frac{\partial a_1}{\partial w_{11}}$$

$$\frac{\partial Loss}{\partial w_{13}} = \frac{\partial Loss}{\partial \hat{y}} \bullet \frac{\partial \hat{y}}{\partial a_1} \bullet \frac{\partial a_1}{\partial w_{13}}$$

$$\frac{\partial Loss}{\partial b_{11}} = \frac{\partial Loss}{\partial \hat{y}} \bullet \frac{\partial \hat{y}}{\partial a_1} \bullet \frac{\partial a_1}{\partial b_{11}}$$



Backpropagation

Backpropagation of DNN

Step 5: Update all weights

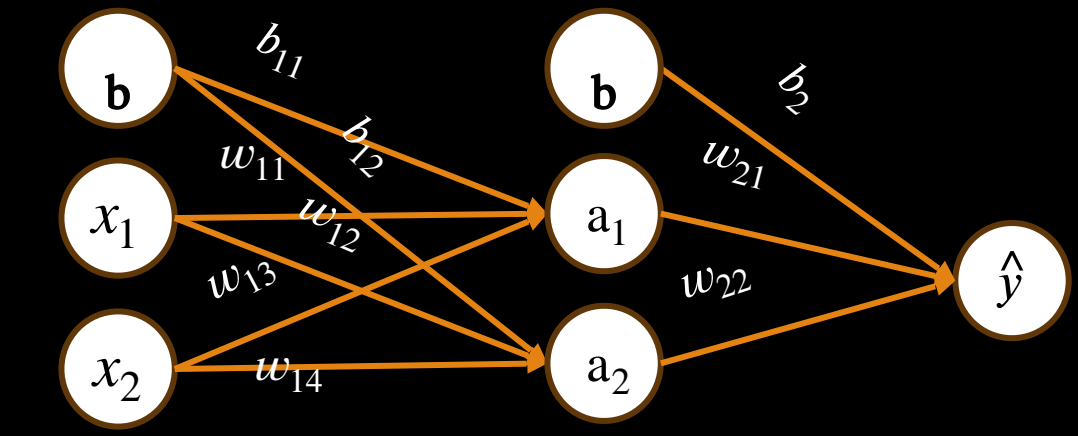
$$w_{11} = w_{11} - \alpha \bullet \frac{\partial Loss}{\partial w_{11}} \qquad w_{21} = w_{21} - \alpha \bullet \frac{\partial Loss}{\partial w_{21}}$$

$$w_{12} = w_{12} - \alpha \bullet \frac{\partial Loss}{\partial w_{12}} \qquad w_{22} = w_{22} - \alpha \bullet \frac{\partial Loss}{\partial w_{22}}$$

$$w_{13} = w_{13} - \alpha \bullet \frac{\partial Loss}{\partial w_{13}} \qquad b_{11} = b_{11} - \alpha \bullet \frac{\partial Loss}{\partial b_{11}}$$

$$w_{14} = w_{14} - \alpha \bullet \frac{\partial Loss}{\partial w_{14}} \qquad b_{12} = b_{12} - \alpha \bullet \frac{\partial Loss}{\partial b_{12}}$$

$$b_2 = b_2 - \alpha \bullet \frac{\partial Loss}{\partial b_2}$$

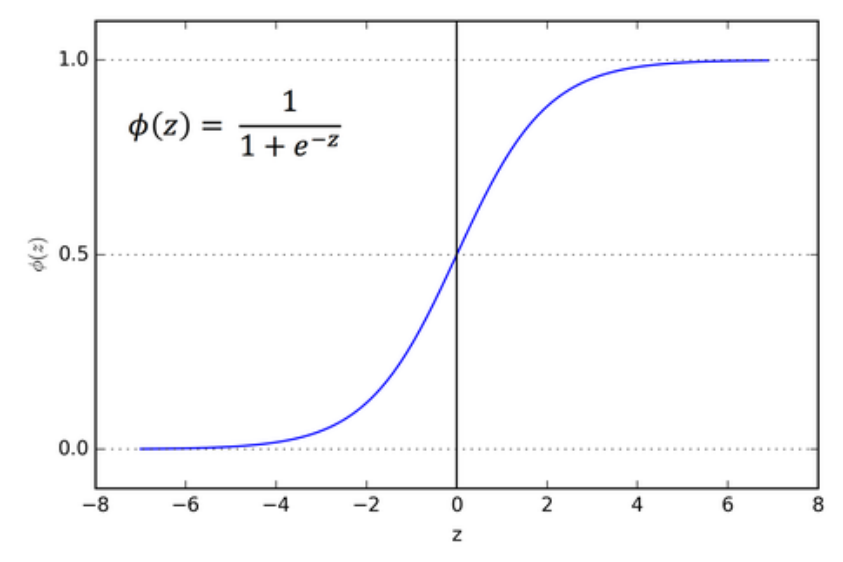


Content

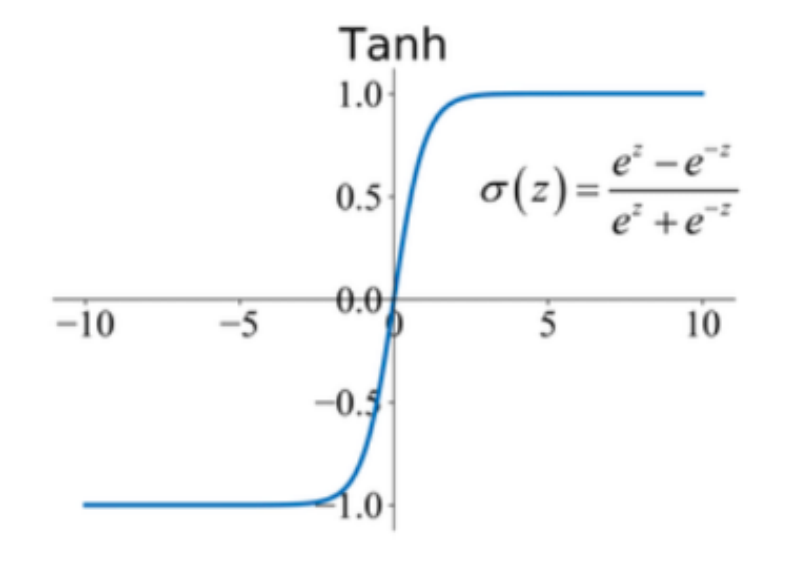
- ~~History and Intro to Deep Learning~~
- ~~Model representation of Artificial Neural Network~~
- ~~Loss Functions of Neural Network~~
- ~~Backpropagation~~
- Activation Functions
- Implementation of DNN

Activation Functions

Sigmoid function

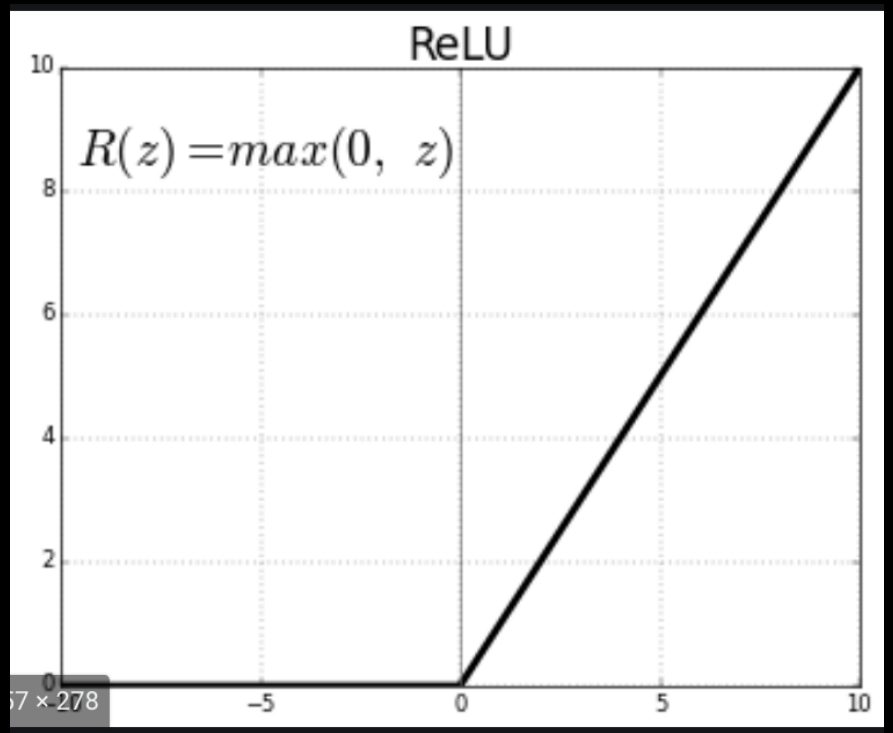


Tanh function

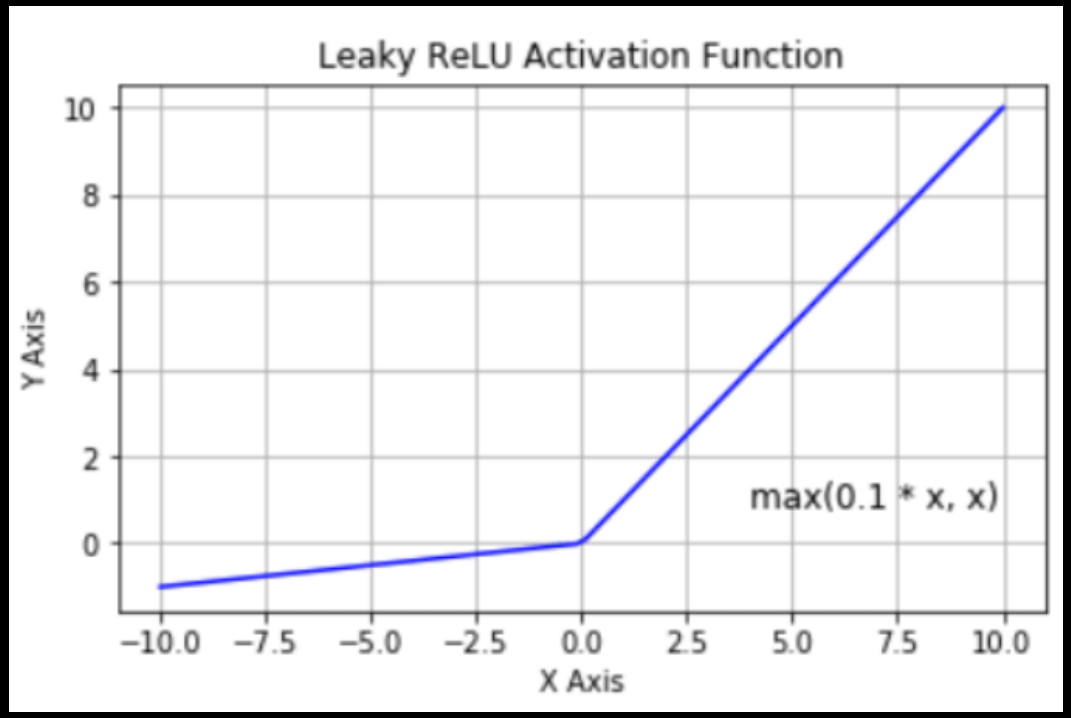


Activation Functions

Rectified Linear Unit (ReLU) function

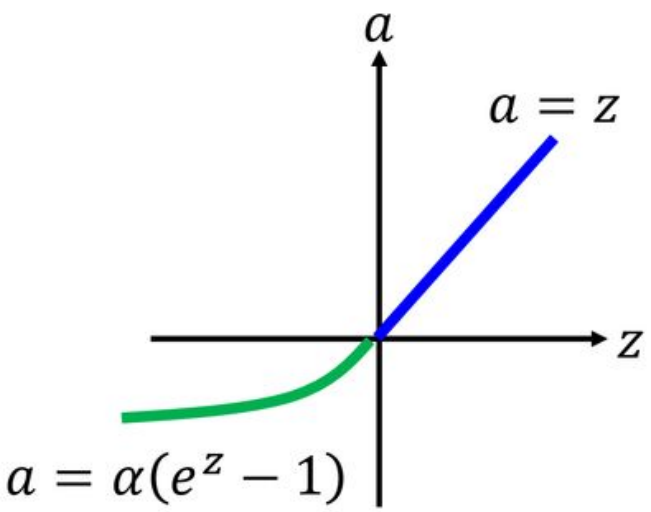


Leaky ReLU function

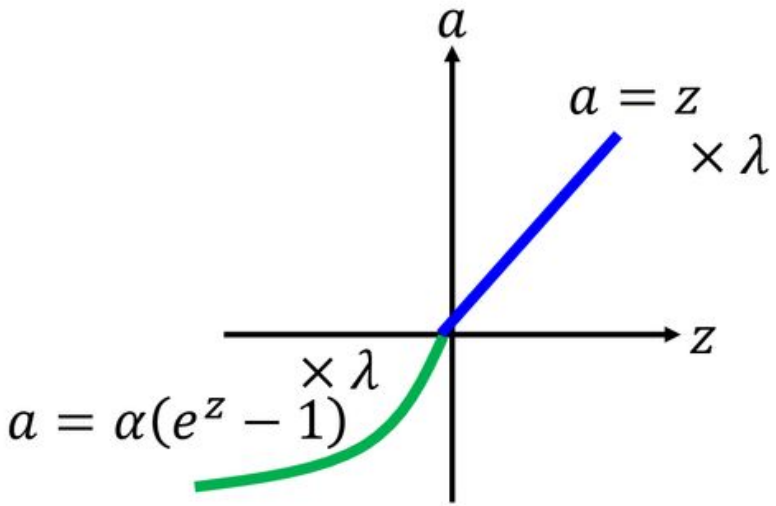


Activation Functions

Exponential Linear Unit (ELU)



Scaled ELU (SELU)



$$\alpha = 1.67326324235437728481704299167 17$$

$$\lambda = 1.05070098735548049341933498529 46$$

Content

- ~~History and Intro to Deep Learning~~
- ~~Model representation of Artificial Neural Network~~
- ~~Loss Functions of Neural Network~~
- ~~Backpropagation~~
- ~~Activation Functions~~
- Implementation of DNN




THANK YOU